

Grocylink

Dokumentation

Version: 1.5.0

Datum: 2026-08-09

Autor: c42u

Co-Autor: ClaudeCode

Lizenz: GPLv3

Grocylink - Dokumentation

Grocylink ist eine Webapplikation, die den Lagerbestand einer [Grocy](#)-Instanz ueberwacht und automatisch Benachrichtigungen versendet, wenn Produkte bald ablaufen, bereits abgelaufen sind oder der Mindestbestand unterschritten wird. Zusaetzhlich synchronisiert Grocylink Tasks und Chores bidirektional mit einem CalDAV-Server, schiebt benoetigte Eintraege auf eine **Bring!-Einkaufsliste** und kann PDF-Kassenbons einlesen, Produkte extrahieren und als Bestand buchen.

Inhaltsverzeichnis

- [Features](#)
 - [Architektur](#)
 - [Installation](#)
 - [Docker-Netzwerk einrichten](#)
 - [Docker \(empfohlen\)](#)
 - [Komodo](#)
 - [Manuell](#)
 - [Konfiguration](#)
 - [Grocy-Verbindung](#)
 - [Benachrichtigungskanaele](#)
 - [Individuelle Warntage](#)
 - [CalDAV-Synchronisation](#)
 - [Bring!-Synchronisation](#)
 - [Kassenbon-Scanner](#)
 - [Reverse Proxy / HTTPS](#)
 - [Sicherheit](#)
 - [API-Referenz](#)
 - [Fehlerbehebung](#)
-

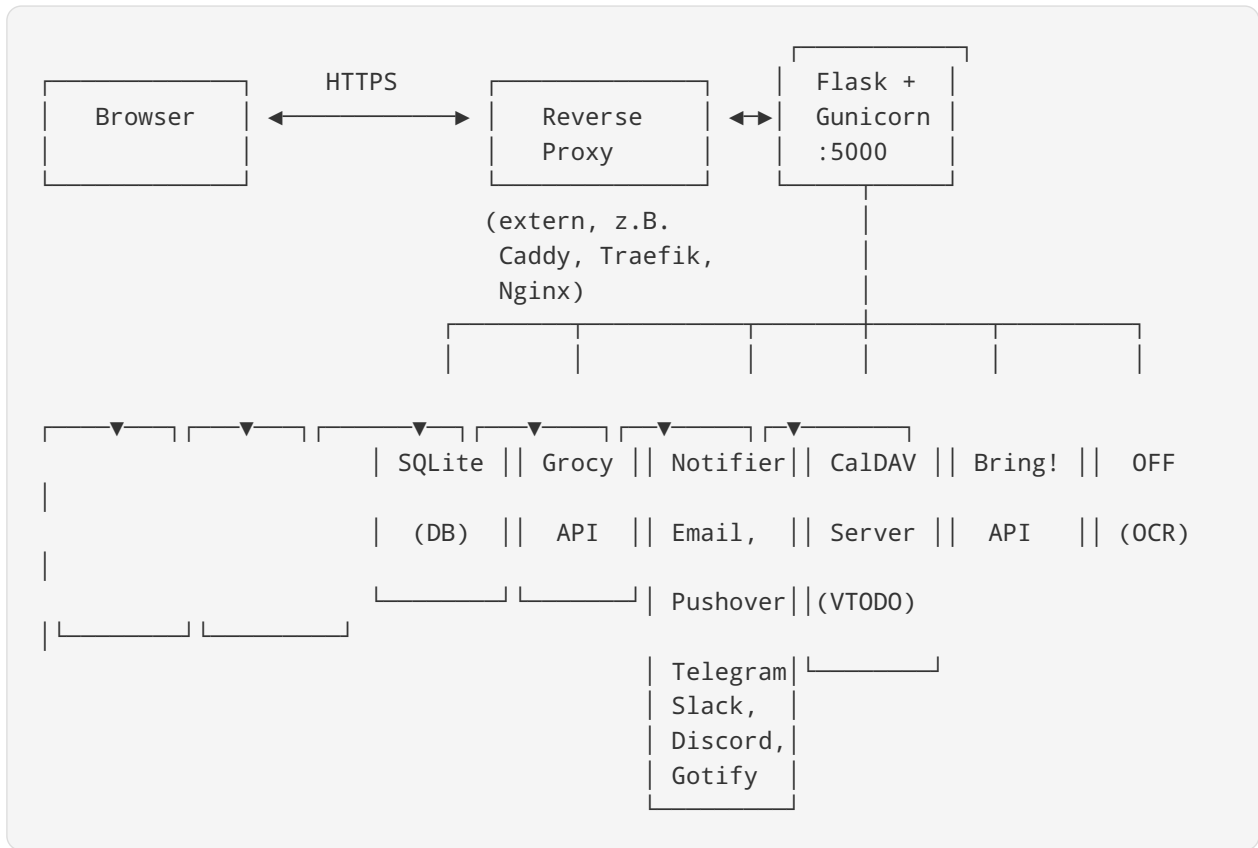
Features

- **Dashboard** mit Echtzeit-Uebersicht ueber ablaufende, abgelaufene und fehlende Produkte
 - **6 Benachrichtigungskanaele:** Email (SMTP), Pushover, Telegram, Slack, Discord, Gotify
-



-
- **Individuelle Warntage** pro Produkt konfigurierbar (z.B. Milch 2 Tage, Konserven 30 Tage)
 - **Automatischer Scheduler** mit konfigurierbarem Intervall (Standard: 6 Stunden)
 - **Manueller Check** per Button-Klick
 - **Test-Funktion** fuer jeden Notification-Kanal
 - **Benachrichtigungsprotokoll** mit vollstaendiger Historie
 - **Dark/Light Mode** (automatisch via System-Einstellung + manueller Toggle)
 - **Responsives Design** fuer Desktop und Mobilgeraete
 - **Verschlüsselte Speicherung** aller Passwoerter und API-Keys
 - **CalDAV-Synchronisation**: Bidirektionale Sync von Grocy Tasks und Chores als VTOD-Entries
 - **Bring!-Synchronisation**: Grocy-Einkaufsliste oder Mindestbestand-Lueckenliste auf eine Bring!-Liste schieben, mit Auto-Remove und Per-Produkt-Overrides
 - **Kassenbon-Scanner**: PDF-Kassenbons hochladen oder per Ordnerueberwachung einlesen, Produkte und Preise per OCR extrahieren, automatisch Grocy-Produkten zuordnen und als Bestand buchen
 - **Mehrsprachig**: Deutsch und Englisch
 - **Non-Root Container**: Laeuft ohne Root-Rechte mit minimalen Berechtigungen
-

Architektur



Projektstruktur

```
grocy/
├── compose.yaml          # Docker Compose
├── app.py                # Flask-Server, API-Routen, Scheduler-Init
├── database.py           # SQLite-Zugriff mit Verschlüsselung
├── grocy_client.py       # Grocy REST-API Client
├── notifiers.py          # Notification-Provider (6 Stueck)
├── scheduler.py          # Automatische Check-Logik
├── caldav_sync.py        # CalDAV-Synchronisation (VTODO bidirektional)
├── bring_sync.py         # Bring!-Synchronisation (Grocy → Bring, async)
├── bring_runtime.py       # Dauerhafter Eventloop + gecachter Bring-Client
├── receipt_scanner.py    # Kassenbon-Pipeline (OCR, Parsing, Matching)
├── crypto.py             # Fernet-Verschlüsselung fuer sensible Daten
├── requirements.txt       # Python-Dependencies
├── requirements-dev.txt   # Zusaetzlich fuer die Tests (pytest)
├── pytest.ini            # Testkonfiguration
├── .dockerignore
├── tests/                # Testsuite (nicht im Docker-Image)
│   ├── conftest.py       # Attrappen fuer Bring-Client und Antworttypen
│   ├── test_bring_runtime.py
│   └── test_bring_sync.py
├── templates/
│   └── index.html        # Single-Page Frontend
├── static/
│   ├── style.css         # Design mit Dark/Light Mode
│   ├── i18n.js           # Internationalisierung (DE/EN)
│   └── app.js            # Frontend-Logik
├── docker/
│   ├── Dockerfile
│   ├── .env              # Umgebungsvariablen (IP, Pfade, Timezone)
│   └── entrypoint.sh     # Gunicorn Start
├── data/                 # SQLite DB + Encryption Key (automatisch erstellt)
└── doku/
    └── README.md         # Diese Dokumentation
```

Installation

Docker-Netzwerk einrichten

Grocylink verwendet ein externes Docker-Netzwerk mit festen IP-Adressen. Das Netzwerk muss **einmalig** vor dem ersten Start erstellt werden.

Macvlan-Netzwerk (empfohlen fuer dedizierte IPs)

Ein **Macvlan-Netzwerk** gibt jedem Container eine eigene IP-Adresse im physischen Netzwerk. Die Container sind damit direkt ueber das LAN erreichbar - wie ein eigenstaendiger Server.

```
docker network create -d macvlan \
  --subnet=192.168.0.0/16 \
  --gateway=192.168.255.254 \
  -o parent=ens19 \
  DockerNetwork
```

Parameter	Beschreibung
<code>-d macvlan</code>	Treiber: Macvlan - Container erhalten eigene MAC-Adressen im physischen Netzwerk
<code>--subnet=192.168.0.0/16</code>	Subnetz des physischen Netzwerks. Muss zum bestehenden LAN passen. <code>/16</code> erlaubt Adressen von <code>192.168.0.1</code> bis <code>192.168.255.254</code>
<code>--gateway=192.168.255.254</code>	Standard-Gateway (Router) des Netzwerks. In den meisten Heimnetzwerken die Router-IP
<code>-o parent=ens19</code>	Physisches Netzwerk-Interface des Hosts. Mit <code>ip link show</code> prüfen (häufig: <code>eth0</code> , <code>ens18</code> , <code>ens19</code> , <code>enp0s3</code>)
<code>DockerNetwork</code>	Name des Netzwerks - muss mit dem Namen in <code>compose.yaml</code> uebereinstimmen

Wichtig: Das Subnetz und Gateway muessen zum bestehenden Netzwerk passen. Beispiele: - Heimnetzwerk mit Fritz!Box: `--subnet=192.168.178.0/24 --gateway=192.168.178.1` - Proxmox/Server-Umgebung: Subnetz und Gateway je nach VLAN-Konfiguration anpassen - Das physische Interface (`-o parent=...`) mit `ip link show` oder `ip addr` ermitteln

Macvlan-Einschraenkung: Der Docker-Host selbst kann Container im Macvlan-Netzwerk **nicht direkt** erreichen (und umgekehrt). Falls der Host auf den Container zugreifen muss, eine Macvlan-Subinterface-Bridge einrichten oder einen separaten Proxy verwenden.

Bridge-Netzwerk (Alternative)

Falls kein Macvlan gewuenscht ist, kann auch ein normales Bridge-Netzwerk mit festem Subnetz verwendet werden:



```
docker network create \  
  --subnet=10.0.50.0/24 \  
  --gateway=10.0.50.1 \  
  DockerNetwork
```

Bei einem Bridge-Netzwerk sind Container nur ueber Port-Mappings (`ports:`) von aussen erreichbar.

Netzwerk pruefen

```
# Netzwerk anzeigen  
docker network inspect DockerNetwork  
  
# Alle Netzwerke auflisten  
docker network ls
```

Docker (empfohlen)

Voraussetzungen

- Docker und Docker Compose installiert
- Docker-Netzwerk `DockerNetwork` erstellt (siehe oben)
- Quellcode auf dem Server vorhanden
- **Externer Reverse Proxy** fuer HTTPS (z.B. Caddy, Traefik, Nginx)

Image bauen und starten

```
cd /pfad/zugrocylink  
  
# Image bauen  
docker build --no-cache --network host -t grocylink:latest -f docker/Dockerfile .  
  
# Container starten  
docker compose up -d  
  
# Logs anzeigen  
docker compose logs -f
```

Die Anwendung ist dann erreichbar unter `http://<GROCYLINK_IP>:5000` . Fuer HTTPS einen Reverse Proxy vorschalten (siehe [Reverse Proxy / HTTPS](#)).

Umgebungsvariablen (.env)

Die Datei `docker/.env` enthaelt alle konfigurierbaren Umgebungsvariablen:



```
GUNICORN_WORKERS=2
TIMEZONE=Europe/Berlin
GROCYLINK_IP=10.0.50.26
PATH_TO=/opt/docker-data
UID=1000
GID=1000
```

Variable	Standard	Beschreibung
<code>GUNICORN_WORKERS</code>	<code>2</code>	Anzahl der Unicorn Worker-Prozesse
<code>TIMEZONE</code>	<code>Europe/Berlin</code>	Zeitzone fuer Scheduler und Logs
<code>GROCYLINK_IP</code>	<code>10.0.50.26</code>	Feste IP-Adresse im Docker-Netzwerk
<code>PATH_TO</code>	<code>/opt/docker-data</code>	Basispfad fuer persistente Daten (Bind Mounts)
<code>UID</code>	<code>1000</code>	User-ID unter der der Container laeuft
<code>GID</code>	<code>1000</code>	Group-ID unter der der Container laeuft

Die Datenverzeichnisse werden automatisch unter `${PATH_TO}/grocylink/` angelegt: - `${PATH_TO}/grocylink/data/` - Datenbank und Encryption Key - `${PATH_TO}/grocylink/receipts/` - Kassenbon-PDFs fuer Ordnerueberwachung

Container-Sicherheit

Der Container laeuft mit minimalen Berechtigungen:

```
user: ${UID}:${GID}          # Non-Root User
cap_drop:
  - ALL                      # Alle Linux Capabilities entfernt
security_opt:
  - no-new-privileges:true   # Keine Privilege Escalation
cpus: "1.0"                 # Max 1 CPU-Kern
mem_limit: 2G               # Max 2 GB RAM
pids_limit: 200             # Max 200 Prozesse
```

Wichtig: Die Datenverzeichnisse muessen fuer den konfigurierten User (UID/GID) schreibbar sein:


```
mkdir -p /opt/docker-data/grocylink/data /opt/docker-data/grocylink/receipts
chown 1000:1000 /opt/docker-data/grocylink/data /opt/docker-data/grocylink/receipts
```

Komodo

Komodo ist eine Self-Hosted Plattform zur Verwaltung von Docker-Containern und Deployments. Grocylink kann als **Stack** in Komodo eingebunden werden - ohne Git, rein ueber Dateien auf dem Host.

Voraussetzungen: - Komodo Core laeuft und ist erreichbar - Komodo Periphery ist auf dem Zielsystem installiert und verbunden - Der Zielsystem hat Docker und Docker Compose installiert

Wichtiger Hinweis: Komodo kann standardmaessig keine lokalen Docker-Images bauen. Daher wird das Image **vorab manuell** auf dem Server gebaut und in der `compose.yaml` als fertiges Image referenziert (`image:` statt `build:`). Komodo startet dann nur den Container.

Schritt 1: Dateien auf den Server kopieren

Zuerst muss der komplette Grocylink-Quellcode auf den Zielsystem uebertragen werden. Das Zielverzeichnis kann frei gewaehlt werden, z.B. `/opt/grocylink`.

Per SCP (von lokalem Rechner):

```
scp -r /pfad/zu/grocy/ benutzer@server:/opt/grocylink
```

Per rsync (effizienter bei Updates):

```
rsync -avz --exclude='data/' /pfad/zu/grocy/ benutzer@server:/opt/grocylink/
```

Oder manuell per SFTP/Dateimanager - z.B. mit FileZilla, WinSCP oder dem Dateimanager des Server-Panels.

Nach dem Kopieren sollte die Verzeichnisstruktur auf dem Server so aussehen:

```
/opt/grocylink/
├── compose.yaml          # Compose fuer lokales docker compose
├── app.py
├── database.py
├── grocy_client.py
├── caldav_sync.py
├── notifiers.py
├── scheduler.py
├── crypto.py
├── requirements.txt
├── templates/
│   └── index.html
├── static/
│   ├── i18n.js          # Internationalisierung (DE/EN)
│   ├── app.js
│   └── style.css
└── docker/
    ├── Dockerfile
    ├── .env              # Umgebungsvariablen
    └── entrypoint.sh
```

Schritt 2: Docker-Image manuell bauen

Das Image muss auf dem Zielsystem manuell gebaut werden, **bevor** Komodo den Stack deployed:

```
cd /opt/grocylink
docker build --no-cache --network host -t grocylink:latest -f docker/Dockerfile .
```

Flag	Erklärung
<code>--no-cache</code>	Erzwingt einen vollständigen Rebuild ohne gecachte Layer. Wichtig nach Code-Änderungen.
<code>--network host</code>	Verwendet das Host-Netzwerk während des Builds. Behebt DNS-Probleme, die in manchen Docker-Umgebungen auftreten (z.B. <code>deb.debian.org</code> nicht auflösbar).
<code>-t grocylink:latest</code>	Taggt das Image als <code>grocylink:latest</code> - dieser Name muss mit der <code>compose.yaml</code> übereinstimmen.
<code>-f docker/Dockerfile .</code>	Verwendet das Dockerfile aus dem <code>docker/</code> -Unterverzeichnis mit dem aktuellen Verzeichnis als Build-Context.

Haeufiges Problem: Falls der Build mit DNS-Fehlern wie `Temporary failure resolving 'deb.debian.org'` fehlschlaegt, ist `--network host` die Loesung. Docker verwendet sonst ein eigenes Netzwerk fuer den Build, das je nach Server-Konfiguration keine DNS-Aufloesung hat.

Schritt 3: Stack in Komodo anlegen

1. In Komodo auf **Stacks** > **+ New Stack** klicken
2. **Name:** `grocylink`
3. **Server** auswaehlen (z.B. `vSrv-Docker01`)

Schritt 4: Compose File in Komodo eintragen

Unter **Compose File** in Komodo den folgenden Inhalt eintragen:

```
services:
  grocylink:
    image: grocylink:latest
    pull_policy: never
    container_name: grocylink
    restart: unless-stopped
    user: ${UID}:${GID}
    cpus: "1.0"
    pids_limit: 200
    mem_limit: 2G
    memswap_limit: 3G
    oom_kill_disable: false
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
    environment:
      GUNICORN_WORKERS: ${GUNICORN_WORKERS}
      TZ: ${TIMEZONE}
    volumes:
      - ${PATH_TO}/grocylink/data:/app/data
      - ${PATH_TO}/grocylink/receipts:/app/receipts
    ports:
      - "5000:5000"
    networks:
      DockerNetwork:
        ipv4_address: ${GROCYLINK_IP}

networks:
  DockerNetwork:
    external: true
```

Feld	Erklaerung
<code>image: grocylink:latest</code>	Referenziert das lokal gebaute Image (aus Schritt 2).
<code>pull_policy: never</code>	Wichtig! Verhindert, dass Docker/Komodo versucht, das Image von Docker Hub zu pullen. Ohne dieses Flag schlaegt der Deploy mit <code>pull access denied</code> fehl.
<code>user: \${UID}:\${GID}</code>	Container laeuft als Non-Root User.
<code>cap_drop: ALL</code>	Alle Linux Capabilities werden entfernt.
<code>security_opt: no-new-privileges</code>	Verhindert Privilege Escalation.
<code>volumes: \${PATH_T0}/...:/app/data</code>	Bind Mount fuer Datenbank und Encryption Key.
<code>volumes: \${PATH_T0}/...:/app/receipts</code>	Bind Mount fuer Kassenbon-PDFs (Ordnerueberwachung).
<code>ports: "5000:5000"</code>	Gunicorn Port. Fuer HTTPS einen Reverse Proxy vorschalten.
<code>networks: DockerNetwork</code>	Externes Docker-Netzwerk mit fester IP (muss vorher erstellt werden, siehe Docker-Netzwerk einrichten).

Schritt 5: Environment konfigurieren

Unter **Environment** in Komodo die Umgebungsvariablen setzen:

```
GUNICORN_WORKERS=2
TIMEZONE=Europe/Berlin
GROCYLINK_IP=10.0.50.26
PATH_T0=/opt/docker-data
UID=1000
GID=1000
```

Variable	Standard	Beschreibung
<code>GUNICORN_WORKERS</code>	<code>2</code>	Anzahl Gunicorn Worker-Prozesse
<code>TIMEZONE</code>	<code>Europe/Berlin</code>	Zeitzone fuer Scheduler und Logs

Variable	Standard	Beschreibung
<code>GROCYLINK_IP</code>	<code>10.0.50.26</code>	Feste IP-Adresse im Docker-Netzwerk. Muss eine freie Adresse im konfigurierten Subnetz sein.
<code>PATH_TO</code>	<code>/opt/docker-data</code>	Basispfad fuer persistente Daten auf dem Host.
<code>UID</code>	<code>1000</code>	User-ID unter der der Container laeuft
<code>GID</code>	<code>1000</code>	Group-ID unter der der Container laeuft

Wichtig: Die IP-Adresse (`GROCYLINK_IP`) muss im Subnetz des Docker-Netzwerks liegen und darf nicht von einem anderen Container oder Geraet belegt sein.

Wichtig: Die Datenverzeichnisse muessen fuer den konfigurierten User schreibbar sein:

```
mkdir -p ${PATH_TO}/grocylink/data ${PATH_TO}/grocylink/receipts
chown ${UID}:${GID} ${PATH_TO}/grocylink/data ${PATH_TO}/grocylink/receipts
```

Schritt 6: Deploy

1. In Komodo auf **Deploy** klicken
2. Komodo fuehrt `docker compose -f compose.yaml up -d` im Run Directory aus
3. Der Container startet mit dem lokal gebauten Image `grocylink:latest`

Nach dem Deployment

- Grocylink ist erreichbar unter `http://<GROCYLINK_IP>:5000` bzw. ueber den vorgeschalteten Reverse Proxy
- In Komodo unter **Stacks** > **grocylink** sind Logs, Container-Status und Ressourcenverbrauch sichtbar
- **Daten** liegen persistent unter `${PATH_TO}/grocylink/data/` auf dem Host

Betrieb hinter einem Reverse Proxy

Grocylink stellt nur HTTP auf Port 5000 bereit. Fuer HTTPS wird ein externer Reverse Proxy empfohlen. Beispiel fuer eine Caddy-Konfiguration:

```
grocylink.example.com {  
    tls /certs/fullchain.pem /certs/key.pem  
    reverse_proxy http://10.0.50.26:5000  
}
```

Wichtig: Beim TLS-Zertifikat immer die **fullchain.pem** (Leaf + Intermediate) verwenden, nicht nur die **cert.pem**. Details dazu im Abschnitt [Reverse Proxy / HTTPS](#).

Updates durchführen

Bei Code-Änderungen müssen Image und Container aktualisiert werden:

```
# 1. Neue Dateien auf den Server kopieren  
rsync -avz --exclude='data/' /pfad/zu/grocy/ benutzer@server:/opt/grocylink/  
  
# 2. Image neu bauen (auf dem Server)  
cd /opt/grocylink  
docker build --no-cache --network host -t grocylink:latest -f docker/Dockerfile .  
  
# 3. In Komodo: Stacks > grocylink > Redeploy
```

Tipp: Das **data/**-Verzeichnis beim Kopieren ausschliessen (**--exclude='data/'**), damit die Datenbank und der Encryption Key nicht ueberschrieben werden. Diese liegen im Docker-Volume und sind vom Quellcode getrennt.

Monitoring in Komodo

Nach dem Deployment zeigt Komodo: - **Container-Status:** Running/Stopped mit Uptime - **Logs:** Echtzeit-Logs von Gunicorn - **Ressourcen:** CPU- und RAM-Verbrauch des Containers - **Actions:** Start, Stop, Restart, Redeploy direkt aus der UI

Manuell

Voraussetzungen: Python 3.10+

```
cd /opt/claude/grocy  
  
# Dependencies installieren  
pip install -r requirements.txt  
  
# App starten (Entwicklungsmodus)  
python3 app.py
```

Die App laeuft dann auf `http://localhost:5000`.

Fuer HTTPS im manuellen Betrieb einen Reverse Proxy (z.B. Caddy, Traefik) vorschalten.

Tests ausfuehren

Die Testsuite fuer den Bring!-Layer laeuft ohne Netzwerk und ohne Datenbank - sie fasst weder die echte SQLite-Datei noch die Bring-API an.

```
cd Code
pip install -r requirements-dev.txt
python3 -m pytest
```

Mit Abdeckungsbericht:

```
python3 -m pytest --cov=bring_sync --cov=bring_runtime --cov-report=term-missing
```

Konfiguration

Grocy-Verbindung

1. Oeffne die Webapplikation im Browser
2. Navigiere zu **Einstellungen**
3. Trage die **Grocy URL** ein (z.B. `https://grocy.example.com`)
4. Trage den **API-Key** ein (in Grocy unter Einstellungen > API-Keys zu finden)
5. Klicke auf **Verbindung testen**
6. Speichere die Einstellungen

Benachrichtigungskanaele

Navigiere zu **Kanaele** und klicke auf **+ Kanal hinzufuegen**.

Email (SMTP)

Feld	Beispiel	Beschreibung
SMTP Host	<code>smtp.gmail.com</code>	SMTP-Server
SMTP Port	<code>587</code>	Port (587 fuer STARTTLS)
Benutzername	<code>user@gmail.com</code>	Login
Passwort	<code>app-passwort</code>	Passwort/App-Passwort



Feld	Beispiel	Beschreibung
Absender-Email	user@gmail.com	Von-Adresse
Empfaenger-Email	notify@example.com	Ziel-Adresse
TLS verwenden	ja	STARTTLS aktivieren

Hinweis Gmail: Es muss ein [App-Passwort](#) verwendet werden.

Pushover

Feld	Beschreibung
API Token	App-Token von pushover.net
User Key	User/Group Key
Prioritaet	-2 (leise) bis 2 (Notfall)

Telegram

Feld	Beschreibung
Bot Token	Token vom BotFather
Chat ID	Ziel-Chat (User-ID oder Gruppen-ID)

Schritt-fuer-Schritt Einrichtung:

1. **Bot erstellen:** In Telegram den [BotFather](#) oeffnen und `/newbot` senden
2. Einen **Namen** und einen **Benutzernamen** fuer den Bot vergeben (Benutzername muss auf `bot` enden, z.B. `MeinGrocyBot`)
3. Der BotFather gibt einen **HTTP API Token** zurueck im Format:

```
1234567890:AABBccDDeeFFggHHiiJJkkLLmmNNooPPqq
```

Diesen Token in Grocylink unter **Bot Token** eintragen.

4. Chat ID ermitteln:

- Den eigenen Bot in Telegram oeffnen (nach dem Benutzernamen suchen)
- Auf **Start** klicken oder eine beliebige Nachricht senden (z.B. "Hallo")
- Folgende URL im Browser aufrufen (Token einsetzen):


```
https://api.telegram.org/bot<DEIN_KOMPLETTER_TOKEN>/getUpdates
```

Der Token ist der **gesamte String** inklusive der Zahl, dem Doppelpunkt und dem Buchstabenteil.

- In der JSON-Antwort die Chat ID ablesen:

```
{
  "ok": true,
  "result": [{
    "message": {
      "chat": {
        "id": 1234567890,
        "first_name": "Max",
        "type": "private"
      }
    }
  }]
}
```

- Die Zahl hinter **"id"** (z.B. **1234567890**) ist die **Chat ID** - diese in Grocylink eintragen.

5. In Grocylink auf **Test** klicken - es sollte eine Testnachricht im Telegram-Chat erscheinen.

Leere Antwort ("result": []): Falls **getUpdates** ein leeres Ergebnis liefert, wurde noch keine Nachricht an den Bot gesendet. Erst im Telegram-Chat eine Nachricht an den Bot schicken, dann die URL erneut aufrufen.

Gruppen-Benachrichtigungen: Um Nachrichten in eine Telegram-Gruppe zu senden, den Bot zur Gruppe hinzufügen, dort eine Nachricht senden, und dann **getUpdates** aufrufen. Gruppen-Chat-IDs sind negativ (z.B. **-100123456789**).

Sicherheitshinweis: Den Bot-Token niemals öffentlich teilen. Falls der Token kompromittiert wurde, beim BotFather mit **/revoke** einen neuen Token generieren.

Slack

Feld	Beschreibung
Webhook URL	Incoming Webhook URL

Webhook erstellen unter: Slack App > Incoming Webhooks

Discord

Feld	Beschreibung
Webhook URL	Discord Channel Webhook URL

Webhook erstellen: Kanal-Einstellungen > Integrationen > Webhooks

Gotify

Feld	Beschreibung
Server URL	Gotify-Serveradresse
App Token	App-Token aus Gotify
Prioritaet	Numerisch (Standard: 5)

Individuelle Warntage

Unter **Produkte** kann fuer jedes Produkt ein individueller Warnzeitraum gesetzt werden:

- **Standard** (aus Einstellungen, z.B. 5 Tage)
- **Individuell** pro Produkt (z.B. Milch = 2 Tage, Konserven = 30 Tage)

Einfach im Feld "Warntage" den gewuenschten Wert eingeben. "Reset" setzt auf den Standardwert zurueck.

CalDAV-Synchronisation

Grocylink kann Tasks und Chores aus Grocy bidirektional mit einem CalDAV-Server synchronisieren. Jeder Task und jede Chore wird als **VTODO-Eintrag** im CalDAV-Kalender angelegt.

Einrichtung

1. Navigiere zu **CalDAV** in der Seitenleiste
2. Trage die **CalDAV Server URL** ein (z.B. `https://nextcloud.example.com`)
3. Waehle den **Servertyp** aus oder trage manuell den **DAV-Pfad** ein (z.B. `/remote.php/dav`)
4. Gib **Benutzername** und **Passwort** ein
5. Klicke auf **Verbindung testen** - bei Erfolg werden die verfuegbaren Kalender angezeigt
6. Klicke auf **Laden** neben der Kalender-Auswahl und waehle den gewuenschten Kalender
7. Setze das **Sync-Intervall** (Standard: 30 Minuten)
8. Aktiviere die **Automatische Synchronisation**

9. Klicke auf **CalDAV-Einstellungen speichern**

Kompatible CalDAV-Server

Server	URL-Format	Pfad
Nextcloud	<code>https://server.example.com</code>	<code>/remote.php/dav</code>
Radicale	<code>https://server.example.com</code>	<code>/radicale/</code>
Baikal	<code>https://server.example.com</code>	<code>/baikal/html/dav.php</code>
iCloud	<code>https://caldav.icloud.com</code>	-
RFC 6764	<code>https://server.example.com</code>	<code>/.well-known/caldav</code>

Sync-Verhalten

Grocy -> CalDAV: - Jeder Grocy-Task und jede Chore wird als VTODO angelegt - UID-Schema: `grocy-task-{id}@grocylink` / `grocy-chore-{id}@grocylink` - Felder: SUMMARY (Name), DESCRIPTION (Beschreibung), DUE (Faelligkeitsdatum), STATUS - Erledigte Tasks werden mit STATUS=COMPLETED synchronisiert

CalDAV -> Grocy: - Wird ein VTODO im CalDAV-Client als erledigt markiert, wird der entsprechende Task/Chore beim naechsten Sync auch in Grocy als erledigt markiert - Wird ein VTODO wieder auf "offen" gesetzt, wird der Task in Grocy ebenfalls wieder geoeffnet (undo) - Aenderungen an Name, Beschreibung und Faelligkeitsdatum werden zurueck nach Grocy synchronisiert - Neue Aufgaben, die im CalDAV-Client erstellt werden, werden automatisch als neue Tasks in Grocy angelegt - **Duplikat-Erkennung:** Beim Import neuer Aufgaben aus CalDAV wird sowohl die Original-UID als auch die Grocylink-UID in der Sync-Map gespeichert, um doppelte Imports zu verhindern. Zusaetzlich wird geprueft, ob in Grocy bereits ein Task mit identischem Namen existiert

Sync-Mapping

Die Tabelle "Sync-Mapping" auf der CalDAV-Seite zeigt alle synchronisierten Eintraege mit Typ, Grocy-ID, CalDAV-UID, aktuellem Status, Sync-Richtung und Zeitpunkt der letzten Synchronisation.

Bring!-Synchronisation

Grocylink kann Eintraege aus Grocy auf eine [Bring!](#)-Einkaufsliste schieben. Die Integration ist als eigener Sync-Layer implementiert (analog zum CalDAV-Sync), nicht als klassischer Notification-Channel - Bring! ist eine zustandsbehaftete Liste mit Items, Mengen und Status, kein Stateless-Push-Kanal.

Wichtig: Bring! Labs bietet keine offizielle, dokumentierte API. Grocylink nutzt die reverse-engineered Library `miauc1/bring-api`, die auch die Basis fuer die offizielle Home-Assistant-Bring!-Integration bildet. Da die App-API von Bring! nicht stabil zugesichert ist, kann sich das Verhalten jederzeit aendern. Grocylink ist nicht mit Bring! Labs AG affiliert.

Setup

1. Im Tab **Bring!** in der Sidebar Bring!-E-Mail und Passwort eintragen
2. **Verbindung testen** - bei Erfolg werden alle verfuegbaren Listen-Namen angezeigt
3. **Listen laden** klicken und im Dropdown die gewuenschte Liste auswaehlen
4. **Quelle** waehlen:
 - `Grocy-Einkaufsliste` (Default): die explizite Grocy-Shoppinglist wird auf Bring! gespiegelt
 - `Mindestbestand unterschritten`: nur Produkte, deren Bestand unter dem konfigurierten Minimum liegt
5. **Sync-Intervall** (Standard 30 Min) und ggf. **Auto-Remove** aktivieren
6. Checkbox **Automatische Synchronisation aktiviert** setzen und **speichern**

Datenfluss (v1: unidirektional Grocy -> Bring!)

```
Grocy (Shoppinglist | Volatile/Missing)
-> Soll-Items aufbauen (Name, Menge, QU)
-> Per-Produkt-Override anwenden (hide_from_bring, custom_name, custom_spec)
-> Bring-Liste einmal laden + Match ueber UUID-Mapping bzw. Name
-> Differenz sammeln (anlegen / aendern / entfernen)
-> gebuendelt senden (batch_update_list, Bloecke zu 50)
-> sync_map aktualisieren (grocy_product_id <-> bring_item_uuid)
```

Bidirektionaler Sync (Items in Bring! abhaken -> Grocy aktualisieren) ist fuer eine Folgeversion vorgesehen.

Verbindungsmodell (seit 1.5.0)

Grocylink arbeitet synchron (Flask + APScheduler), die Bring-Library ist asynchron. Beides verbindet `bring_runtime.py`: ein **dauerhafter Eventloop** in einem Daemon-Thread, in dem aiohttp-Session und eingeloggter Client zwischen den Aufrufen weiterleben.

- **Ein Login statt einem pro Request.** Bis 1.4.x oeffnete jeder Aufruf - Status, Listen, Listeninhalt, Sync - einen eigenen Eventloop mit eigenem Login. Ein Wechsel auf den Bring!-Tab kostete mehrere Anmeldungen.
- **Token-Erneuerung** uebernimmt `bring-api` selbst per Refresh-Token, solange der Client lebt. Laeuft der Token dennoch ab, wiederholt Grocylink den Aufruf einmal mit frischem Login.

- **Credential-Wechsel** wird ueber einen Hash der Zugangsdaten erkannt; beim Speichern neuer Daten wird der gecachte Client sofort verworfen.
- **Zeitlimit** von 180 Sekunden pro Aufruf, damit ein haengender Request keinen Unicorn-Worker dauerhaft belegt.
- Jeder Unicorn-Worker haelt seinen eigenen Client - bei mehreren Workern entsprechend mehrere Sitzungen.

Ein Sync kostet dadurch zwei Requests (Liste holen, Aenderungen senden) statt frueher rund zwei pro Produkt.

Item-Naming

- Der Bring!-Item-Name kommt aus dem Grocy-Produktnamen (oder dem Per-Produkt-Override `custom_name`)
- Bring matcht Item-Namen gegen einen internen Katalog fuer Icons - kanonische Namen wie *Milch*, *Brot* erkennen das System besser als Marken-Bezeichnungen
- Der **Spec** (Detail-Feld) wird als `<Menge> <Mengeneinheit>` gebaut. Plural-Formen aus Grocy werden bei Mengen >1 verwendet
- Das in der Bring-API problematische Zeichen `%` wird automatisch durch `Prozent` ersetzt

Per-Produkt-Overrides

Pro Grocy-Produkt koennen folgende Overrides in der Tabelle `bring_item_overrides` gesetzt werden:

Feld	Bedeutung
<code>hide_from_bring</code>	Produkt wird nicht auf die Bring!-Liste geschoben (Default 0)
<code>custom_name</code>	Abweichender Name auf Bring! (z.B. "Vollmilch" statt "Bio-Milch")
<code>custom_spec</code>	Festes Spec-Feld (uebersteuert die automatische Mengen-Logik)

Verwaltbar via REST-API `POST /api/bring/overrides` .

Auto-Remove

Wenn `bring_auto_remove=1` gesetzt ist, entfernt Grocylink Items, die im **sync_map** stehen, aber im aktuellen Soll-Set nicht mehr vorhanden sind, automatisch von der Bring!-Liste. **Default ist aus** - nutzt Du diese Option, wird die Bring!-Liste effektiv von Grocy autoritativ kontrolliert.

Sync-Mapping

Die Tabelle "Sync-Mapping" auf der Bring!-Seite zeigt: Grocy-Produkt-ID, Bring!-Item-Name, Spec, Bring-UUID und Zeitpunkt der letzten Synchronisation. Ueber **Mapping leeren** kann das Mapping zurueckgesetzt werden, ohne Items in Bring! anzufassen.

Hinweise

- Sparsam pollen - Rate-Limits der Bring-API sind nicht dokumentiert. UI erlaubt minimal 5 Min Intervall
- Beim Umbenennen eines Items legt Grocylink einen neuen Eintrag an und entfernt den alten. Ein reines Update per UUID waere moeglich, die Bring!-App zeigt den geaenderten Namen aber erst nach komplettem Reload an
- Die UUID neuer Eintraege vergibt Grocylink selbst. Das erspart den frueher noetigen Vollabruf der Liste nach jedem einzelnen neuen Item
- Items, die manuell in Bring! abgehakt werden, landen in **recently** und werden bei der naechsten Sync nicht erneut hinzugefuegt, solange sie noch in der **recently**-Liste stehen

Kassenbon-Scanner

Grocylink kann PDF-Kassenbons einlesen, Produkte und Preise extrahieren und als Bestandsbuchungen in Grocy uebernehmen. Sowohl digital erstellte PDFs als auch gescannte Kassenbons (per OCR) werden unterstuetzt.

Verarbeitungs-Pipeline

```
PDF (Upload / Ordnerueberwachung)
  → Typ erkennen (digital / gescannt)
  → Text extrahieren (pdfplumber / Tesseract OCR)
  → Bon parsen (Markt, Datum, Produkte, Preise, Summe)
  → Produkte matchen (gelernte Zuordnung → Fuzzy-Match → kein Treffer)
  → In DB speichern (Status: pending_review)
  → User prueft in UI (Zuordnungen korrigieren/bestaetigen)
  → Bestaetigung → Bestand in Grocy buchen (add_stock)
  → Zuordnungen lernen (fuer zukuenftige Bons)
```

Kassenbons hochladen

1. Navigiere zu **Kassenbons** in der Seitenleiste
2. PDFs per **Drag & Drop** in die Upload-Zone ziehen oder ueber **Dateiauswahl** hochladen
3. Der Kassenbon wird automatisch verarbeitet und erscheint in der Tabelle

Ordnerueberwachung

Grocylink kann einen Ordner periodisch auf neue PDF-Kassenbons ueberwachen:

1. Navigiere zu **Einstellungen > Kassenbons**
2. Trage den **Ueberwachungsordner** ein (Standard: **/app/receipts**)
3. Setze das **Scan-Intervall** (Standard: 5 Minuten)
4. Aktiviere die **Ordnerueberwachung** per Checkbox
5. Speichere die Einstellungen

Neue PDFs im Ordner werden automatisch erkannt, verarbeitet und in die Kassenbon-Liste uebernommen. Bereits verarbeitete Dateien werden anhand des Dateipfads erkannt und nicht erneut eingelesen.

Volume-Mount: Fuer die Ordnerueberwachung muss das Kassenbon-Verzeichnis als Volume gemountet sein:

```
volumes:  
- ${PATH_TO}/grocylink/receipts:/app/receipts
```

Review-Workflow

Nach dem Einlesen erscheint ein Kassenbon mit Status **Ausstehend**:

1. Klicke auf **Pruefen** um den Review-Dialog zu oeffnen
2. Alle erkannten Positionen werden mit ihrer automatischen Zuordnung angezeigt
3. Fuer jede Position wird ein **Match-Score** angezeigt:
 - **Gruen (>90%)**: Hohe Uebereinstimmung
 - **Gelb (70–90%)**: Mittlere Uebereinstimmung — pruefen empfohlen
 - **Rot (<70%)**: Geringe Uebereinstimmung — manuelle Korrektur noetig
4. Zuordnungen per **Dropdown** korrigieren (alle Grocy-Produkte durchsuchbar)
5. Mit **Bestaetigen** werden alle Positionen als Bestand in Grocy gebucht

Bestaetigte Zuordnungen werden automatisch gelernt und bei zukuenftigen Kassenbons als erstes angewendet (exakter Match vor Fuzzy-Match).

Produkt-Matching

Das Matching erfolgt in zwei Stufen:

1. **Gelernte Zuordnungen** (exakt): Zuvor bestaetigte Zuordnungen werden direkt angewendet (Score: 100, Quelle: **learned**)
2. **Fuzzy-Match** (automatisch): Falls keine gelernte Zuordnung existiert, wird per **token_sort_ratio** (rapidfuzz) das aehnlichste Grocy-Produkt gesucht (Quelle: **fuzzy**)

Der **Match-Schwellwert** (Standard: 70%) bestimmt, ab welchem Score ein Fuzzy-Match als Treffer gilt. Produkte unter dem Schwellwert erhalten keine automatische Zuordnung.

Der **Auto-Confirm-Schwellwert** (Standard: 95%) bestimmt, ab welchem Score Zuordnungen automatisch als korrekt gelten.

Gelernte Zuordnungen verwalten

Unter **Kassenbons** > **Gelernte Zuordnungen** (aufklappbar) werden alle gespeicherten Zuordnungen angezeigt. Einzelne Zuordnungen koennen geloescht werden, um das Matching fuer bestimmte Bon-Produkte zurueckzusetzen.

Einstellungen

Einstellung	Standard	Beschreibung
Ueberwachungsordner	/app/receipts	Ordner fuer automatische Bon-Erkennung
Scan-Intervall	5 Minuten	Wie oft der Ordner auf neue PDFs geprueft wird
Match-Schwellwert	70 %	Minimaler Score fuer Fuzzy-Match-Treffer
Auto-Confirm-Schwellwert	95 %	Score ab dem Zuordnungen automatisch als korrekt gelten
Ordnerueberwachung	aus	Aktiviert/deaktiviert die periodische Ordnerueberwachung

Reverse Proxy / HTTPS

Grocylink stellt nur **HTTP auf Port 5000** bereit. Fuer HTTPS wird ein externer Reverse Proxy empfohlen.

Beispiel: Caddy

```
grocylink.example.com {  
    tls /certs/fullchain.pem /certs/key.pem  
    reverse_proxy http://10.0.50.26:5000  
}
```


Beispiel: Nginx

```
server {
    listen 443 ssl;
    server_name grocylink.example.com;

    ssl_certificate /certs/fullchain.pem;
    ssl_certificate_key /certs/key.pem;

    location / {
        proxy_pass http://10.0.50.26:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Beispiel: Traefik (Docker Labels)

```
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.grocylink.rule=Host(`grocylink.example.com`)"
  - "traefik.http.routers.grocylink.tls=true"
  - "traefik.http.services.grocylink.loadbalancer.server.port=5000"
```

Vollstaendige Zertifikatskette

Wichtig: Beim TLS-Zertifikat immer die **fullchain.pem** (Leaf + Intermediate) verwenden, nicht nur die **cert.pem**. Andernfalls koennen Clients (z.B. Python **requests**) die Verbindung nicht verifizieren, da das Intermediate-Zertifikat fehlt.

```
# Fullchain erstellen (Leaf + Intermediate)
cat cert.pem > fullchain.pem
curl -s https://letsencrypt.org/certs/2024/e7.pem >> fullchain.pem
```

Pruefen ob die Datei korrekt ist (muss **2** zurueckgeben):

```
grep -c "BEGIN CERTIFICATE" fullchain.pem
```

Pruefen ob der Server die vollstaendige Chain liefert:

```
openssl s_client -connect domain.example.com:443 -servername
domain.example.com 2>&1 | grep "depth="
# Erwartet: depth=0 (Leaf) und depth=1 (Intermediate)
```

SSL-Verifizierung der Grocy-Verbindung

Grocylink verifiziert standardmaessig das SSL-Zertifikat der Grocy-Instanz. Falls die Grocy-Instanz ein Zertifikat ohne vollstaendige Chain liefert oder ein selbstsigniertes Zertifikat verwendet, kann die Verifizierung unter **Einstellungen** mit der Checkbox **"SSL-Zertifikat verifizieren"** deaktiviert werden.

***Empfehlung:** Die SSL-Verifizierung sollte nur voruebergehend deaktiviert werden. Besser ist es, das SSL-Problem auf dem Grocy-Server zu beheben (vollstaendige Zertifikatskette konfigurieren).*

Sicherheit

Container-Haertung

Grocylink laeuft standardmaessig mit minimalen Berechtigungen:

- **Non-Root:** Container laeuft als unprivilegierter User (konfigurierbar via **UID** / **GID**)
- **cap_drop: ALL:** Alle Linux Capabilities entfernt
- **no-new-privileges:** Keine Privilege Escalation moeglich
- **Ressourcenlimits:** CPU, RAM und Prozessanzahl begrenzt

Verschluesselung sensibler Daten

Alle Passwoerter und API-Keys werden mit **Fernet (AES-128-CBC + HMAC-SHA256)** verschlueselt in der SQLite-Datenbank gespeichert.

Verschlueselte Felder: - Grocy API-Key (Settings) - CalDAV-Passwort (Settings) - Email-Passwoerter (Channel-Config) - API-Tokens: Pushover, Telegram Bot-Token, Gotify App-Token - Webhook-URLs: Slack, Discord

Encryption Key: - Wird automatisch beim ersten Start generiert - Gespeichert in **data/.encryption_key** (Datei-Berechtigung: 600) - **Wichtig:** Diese Datei sichern! Ohne sie koennen verschlueselte Daten nicht entschlueselt werden

Datenpersistenz und Volumes

Grocylink speichert alle Daten in `/app/data/` innerhalb des Containers. Dieses Verzeichnis **muss** als Docker-Volume gemountet sein, damit Daten einen Container-Neustart oder Redeploy ueberleben.

Kritische Dateien in `/app/data/`:

Datei	Beschreibung
<code>grocy_notify.db</code>	SQLite-Datenbank mit allen Einstellungen, Kanälen, Overrides und Logs
<code>.encryption_key</code>	Fernet-Schlüssel zur Ent-/Verschlüsselung aller Passwörter und API-Keys

WICHTIG: Geht der `.encryption_key` verloren (z.B. durch Löschen des Volumes), können alle verschlüsselten Werte in der Datenbank **nicht mehr entschlüsselt** werden. Alle Zugangsdaten (API-Keys, Passwörter, Webhook-URLs, Kanalconfigurationen) müssen dann neu eingegeben werden.

Volume-Konfiguration (Bind Mount):

```
volumes:
  - ${PATH_TO}/grocylink/data:/app/data          # Datenbank + Encryption Key
  - ${PATH_TO}/grocylink/receipts:/app/receipts  # Kassenbon-PDFs
```

Bei der Standard-Konfiguration mit `PATH_TO=/opt/docker-data` liegen die Daten unter: - `/opt/docker-data/grocylink/data/` - Datenbank und Encryption Key - `/opt/docker-data/grocylink/receipts/` - Kassenbon-PDFs fuer Ordnerueberwachung

Beim Redeploy beachten: - Bind Mounts bleiben bei `docker compose down` **immer** erhalten (anders als Named Volumes) - Auch ein Redeploy in Komodo behält alle Daten, da sie direkt auf dem Host liegen - Die Daten bleiben selbst bei `docker compose down -v` erhalten, da `-v` nur Named Volumes löscht - Beim Start prüft Grocylink automatisch ob das Volume korrekt gemountet ist und gibt eine Warnung aus, falls nicht

Integritätsprüfung: Grocylink prüft beim Start ob der Encryption Key zur Datenbank passt. Falls nicht (z.B. nach Key-Verlust), erscheint eine Warnung im Log:

WARNUNG: Encryption Key passt nicht zur Datenbank!
Verschlüsselte Werte können nicht entschlüsselt werden.
Bitte alle Zugangsdaten neu eingeben.

Empfehlungen

- Datenverzeichnis (`data`) regelmässig sichern
- **Backup des Encryption Keys:** `docker cp grocylink:/app/data/.encryption_key ./encryption_key.backup`
- **Backup der Datenbank:** `docker cp grocylink:/app/data/grocy_notify.db ./grocy_notify.db.backup`
- Zugriff auf die Webapplikation via Firewall oder VPN einschränken
- `data/.encryption_key` separat und sicher aufbewahren
- Externen Reverse Proxy fuer HTTPS verwenden

API-Referenz

Alle API-Endpunkte unter `/api/` :

Methode	Pfad	Beschreibung
GET	<code>/api/settings</code>	Einstellungen abrufen
POST	<code>/api/settings</code>	Einstellungen speichern
POST	<code>/api/test-connection</code>	Grocy-Verbindung testen
GET	<code>/api/status</code>	Aktueller Stock-Status (volatile)
GET	<code>/api/channels</code>	Alle Notification-Kanaele
POST	<code>/api/channels</code>	Kanal erstellen/bearbeiten
DELETE	<code>/api/channels/<id></code>	Kanal löschen
POST	<code>/api/channels/<id>/test</code>	Test-Nachricht senden
GET	<code>/api/products</code>	Produkte mit Override-Einstellungen
POST	<code>/api/products/override</code>	Produkt-Warntage setzen
GET	<code>/api/log</code>	Benachrichtigungs-Historie
DELETE	<code>/api/log</code>	Log leeren
POST	<code>/api/check-now</code>	Manuellen Check auslösen
GET	<code>/api/calDav/status</code>	CalDAV-Sync-Status und Statistiken

Methode	Pfad	Beschreibung
POST	/api/caldav/test	CalDAV-Verbindung testen
GET	/api/caldav/calendars	Verfuegbare Kalender abrufen
POST	/api/caldav/sync-now	Manuelle Synchronisation ausloesen
GET	/api/caldav/map	Sync-Mapping-Tabelle abrufen
GET	/api/bring/status	Bring!-Sync-Status und Statistiken
POST	/api/bring/test	Bring!-Login testen
GET	/api/bring/lists	Verfuegbare Bring!-Listen abrufen
POST	/api/bring/sync-now	Manuelle Bring!-Synchronisation ausloesen
GET	/api/bring/map	Bring!-Sync-Mapping abrufen
DELETE	/api/bring/map	Bring!-Sync-Mapping leeren
GET	/api/bring/overrides	Per-Produkt-Overrides abrufen
POST	/api/bring/overrides	Per-Produkt-Override setzen/loeschen
GET	/api/receipts	Alle Kassenbons auflisten
GET	/api/receipts/<id>	Einzelner Kassenbon mit Items
POST	/api/receipts/upload	PDF-Kassenbon hochladen (multipart/form-data)
DELETE	/api/receipts/<id>	Kassenbon loeschen
PUT	/api/receipts/<id>/items/<item_id>	Item-Zuordnung aendern
POST	/api/receipts/<id>/confirm	Kassenbon bestaetigen und Bestand buchen
POST	/api/receipts/<id>/reject	Kassenbon verwerfen
GET	/api/receipts/mappings	Gelernte Zuordnungen abrufen

Methode	Pfad	Beschreibung
DELETE	/api/receipts/mappings/ <id>	Gelernte Zuordnung loeschen
POST	/api/receipts/reprocess/ <id>	Kassenbon erneut matchen

Fehlerbehebung

“Grocy nicht konfiguriert”

Unter Einstellungen muessen Grocy-URL und API-Key eingetragen werden.

Verbindungstest schlaegt fehl

- Ist die Grocy-URL korrekt (inkl. Protokoll `http://` oder `https://`)?
- Ist der API-Key gueltig? (In Grocy pruefen)
- Ist Grocy vom Container/Server aus erreichbar?
- Bei HTTPS: Liefert der Grocy-Server die vollstaendige Zertifikatskette? (siehe [Vollstaendige Zertifikatskette](#))
- Temporaerer Workaround: “SSL-Zertifikat verifizieren” in den Einstellungen deaktivieren

Benachrichtigungen kommen nicht an

1. Kanal-Test-Funktion nutzen (Button “Test”)
2. Log pruefen (Seite “Log”)
3. Bei Email: App-Passwort statt normalem Passwort verwenden (Gmail, etc.)
4. Bei Telegram: Chat-ID pruefen (muss numerisch sein)

SSL-Zertifikat Probleme

Grocy-Verbindung schlaegt mit SSL-Fehler fehl (`CERTIFICATE_VERIFY_FAILED`):

Dieses Problem tritt auf, wenn der Grocy-Server nur das Leaf-Zertifikat liefert, aber nicht die vollstaendige Zertifikatskette (Intermediate fehlt). Python `requests` kann die Kette dann nicht verifizieren.

Diagnose:

```
# Pruefen ob der Server die vollstaendige Chain liefert
openssl s_client -connect grocy.example.com:443 -servername grocy.example.com
2>&1 | grep "depth="
# Nur "depth=0" = Intermediate fehlt!
# "depth=0" + "depth=1" = Chain vollstaendig
```

Loesung (auf dem Grocy-Server): 1. `fullchain.pem` erstellen (Leaf + Intermediate): `bash cat cert.pem > fullchain.pem curl -s https://letsencrypt.org/certs/2024/e7.pem >> fullchain.pem` 2. Reverse Proxy konfigurieren, `fullchain.pem` statt `cert.pem` zu verwenden 3. Berechtigungen pruefen - der Webserver-User muss die Datei lesen koennen 4. Webserver neu starten

Workaround (in Grocylink): Unter Einstellungen die Checkbox "SSL-Zertifikat verifizieren" deaktivieren. Dies sollte nur als temporaere Loesung verwendet werden.

CalDAV-Sync funktioniert nicht

1. **Verbindung testen:** Auf der CalDAV-Seite "Verbindung testen" klicken
2. **URL pruefen:** Die CalDAV-URL muss den korrekten Servertyp/Pfad enthalten (z.B. `/remote.php/dav` bei Nextcloud)
3. **Kalender pruefen:** Ein Kalender muss ausgewaehlt und gespeichert sein
4. **Benutzername/Passwort:** Bei Nextcloud ggf. ein App-Passwort verwenden
5. **Firewall:** Der CalDAV-Server muss vom Container aus erreichbar sein
6. **Log pruefen:** Im Container-Log (`docker compose logs -f`) erscheinen detaillierte Sync-Meldungen

Bring!-Sync funktioniert nicht

1. **Verbindung testen:** Im Tab "Bring!" auf "Verbindung testen" klicken. Bei Erfolg werden alle Listen-Namen ausgegeben.
2. **Login-Daten:** Bring!-E-Mail und Passwort eingeben - Bring! akzeptiert nur Mail/Passwort, kein Token. Bei aktivierter Zwei-Faktor-Authentifizierung im Bring!-Konto kann der Login fehlschlagen.
3. **Liste auswaehlen:** Vor dem Speichern muss eine Liste im Dropdown ausgewaehlt sein. **Listen laden** klicken, dann auswaehlen.
4. **Quelle pruefen:** Bei Quelle `Grocy-Einkaufsliste` muss die Liste in Grocy auch Eintraege haben. Bei Quelle `Mindestbestand` brauchen Produkte einen `min_stock_amount` > 0.
5. **Inoffizielle API:** Sollte sich die Bring-API geaendert haben, kann es zu Library-Fehlern kommen. In dem Fall `bring-api` in `requirements.txt` auf eine neuere Version anheben oder im `notification_log` (Typ `bring_sync`) den Fehler einsehen.
6. **Nach Passwortwechsel:** Grocylink haelt den eingeloggten Client vor. Beim Speichern neuer Zugangsdaten wird er automatisch verworfen; im Zweifel den Container neu starten.

7. **“Bring-Anfrage ohne Antwort abgebrochen”**: Die Bring-API hat innerhalb von 180 Sekunden nicht geantwortet. Meist ein Netzwerkproblem oder eine Störung bei Bring! - der nächste geplante Sync versucht es erneut.

Kassenbon-Scanner erkennt keine Produkte

1. **Digitaler PDF**: Prüfen ob der Bon tatsächlich Textdaten enthält (nicht nur ein eingebettetes Bild). Im Container-Log wird die Extraktionsmethode angezeigt (`digital` oder `ocr`).
2. **Gescannter PDF**: Tesseract OCR benötigt eine ausreichende Bildqualität. Unscharfe oder schief gescannte Bons liefern schlechte Ergebnisse.
3. **Unbekanntes Bon-Format**: Das Parsing unterstützt gängige deutsche Kassenbon-Formate. Bei unbekannten Layouts werden möglicherweise keine Positionen erkannt. Im Container-Log erscheint der extrahierte Rohtext zur Diagnose.

Kassenbon-Ordnerüberwachung funktioniert nicht

1. **Volume prüfen**: Ist `/app/receipts` als Volume gemountet? (`docker inspect grocylink | grep receipts`)
2. **Einstellungen prüfen**: Ordnerüberwachung muss in den Einstellungen aktiviert sein
3. **Berechtigungen**: Das Kassenbon-Verzeichnis muss für den Container-User lesbar sein:

```
chown -R ${UID}:${GID} ${PATH_TO}/grocylink/receipts/
```

4. **Bereits verarbeitet**: Dateien die schon einmal eingelesen wurden, werden nicht erneut verarbeitet (Duplikat-Erkennung per Dateipfad)

Berechtigungsprobleme (Non-Root Container)

Falls der Container mit Berechtigungsfehlern startet:

```
# Datenverzeichnisse für den Container-User schreibbar machen
chown -R 1000:1000 /opt/docker-data/grocylink/data/
chown -R 1000:1000 /opt/docker-data/grocylink/receipts/

# Oder mit der konfigurierten UID/GID
chown -R ${UID}:${GID} ${PATH_TO}/grocylink/data/
chown -R ${UID}:${GID} ${PATH_TO}/grocylink/receipts/
```

Daten nach Redeploy verloren

Symptome: Nach einem Redeploy sind alle Einstellungen, Kanäle und API-Keys weg, oder Kanäle sind vorhanden aber Verbindungstests schlagen fehl.



Ursache 1: Volume nicht gemountet Prüfen ob das Volume korrekt gemountet ist:

```
docker inspect grocylink | grep -A5 "Mounts"
```

Die Ausgabe muss `/app/data` als Mount zeigen. Falls nicht: `compose.yaml` prüfen ob die Volumes konfiguriert sind.

Ursache 2: Encryption Key verloren Falls die DB vorhanden ist, aber verschlüsselte Daten nicht mehr lesbar sind (API-Keys, Passwoerter funktionieren nicht):

```
docker logs grocylink 2>&1 | grep -i "encryption\|warnung"
```

Falls die Warnung "Encryption Key passt nicht" erscheint, muessen alle Zugangsdaten neu eingegeben werden. Ein Backup des Keys kann wiederhergestellt werden:

```
docker cp ./encryption_key.backup grocylink:/app/data/.encryption_key  
docker compose restart
```

Datenbank zuruecksetzen

```
docker exec grocylink rm /app/data/grocy_notify.db  
docker compose restart
```

Achtung: Alle Einstellungen und Kanaele gehen verloren.

Bugs melden & Features vorschlagen

Grocylink verwendet [GitHub Issues](#) als zentralen Bugtracker und Feature-Tracker.

Bug melden

1. Oeffne [Bug Report](#)
2. Beschreibe den Fehler, die Schritte zur Reproduktion und das erwartete Verhalten
3. Fuege wenn moeglich Logs bei (`docker compose logs grocylink`)

Feature vorschlagen

1. Oeffne [Feature Request](#)
2. Beschreibe das gewuenschte Feature und den Anwendungsfall
3. Optional: Loesungsvorschlaege oder Mockups beifuegen



Direkt aus der App

Auf der **Hilfe**-Seite und im **Footer** der App befinden sich direkte Links zu GitHub Issues, um Bugs und Feature Requests schnell einzureichen.